

GraphQL

Guía de arranque: entenderlo, instalarlo y tener algo funcionando en una tarde

Tecnología	GraphQL con Apollo Server 5 (Node.js)
Autor/a	Moisés Flórez Vega
Curso	2º DAW
Fecha	13 de octubre de 2026
Repositorio	https://github.com/MoisesFlorezDAW/manual-graphql-florez

2. ¿Qué es y para qué sirve?

2.1 Definición

GraphQL es un **lenguaje de consulta para APIs** y, a la vez, un **motor para ejecutar esas consultas**. En lugar de tener muchas direcciones fijas que devuelven datos ya empaquetados (como en una API REST), el servidor GraphQL expone **un único punto de entrada** y el cliente escribe exactamente qué datos quiere. El servidor responde con un JSON que tiene la misma forma que la petición, ni más ni menos.

Una analogía: una API REST es un menú cerrado, con platos fijos. GraphQL es una carta en la que eliges los ingredientes que quieres y te sirven solo eso.

2.2 Qué problema resuelve y por qué surgió

Con REST es habitual encontrarse con dos problemas. El primero es el **exceso de datos** (*over-fetching*): el endpoint devuelve 30 campos y la pantalla solo usa 3. El segundo es la **falta de datos** (*under-fetching*): para pintar una pantalla hay que encadenar varias peticiones (primero el libro, luego su autor, luego los otros libros del autor).

GraphQL nació en Facebook alrededor de 2012, cuando renovaron su aplicación móvil. En móvil cada petición y cada byte cuestan, y las pantallas necesitaban datos muy variados que los endpoints fijos no servían bien. Facebook lo publicó como código abierto en 2015.

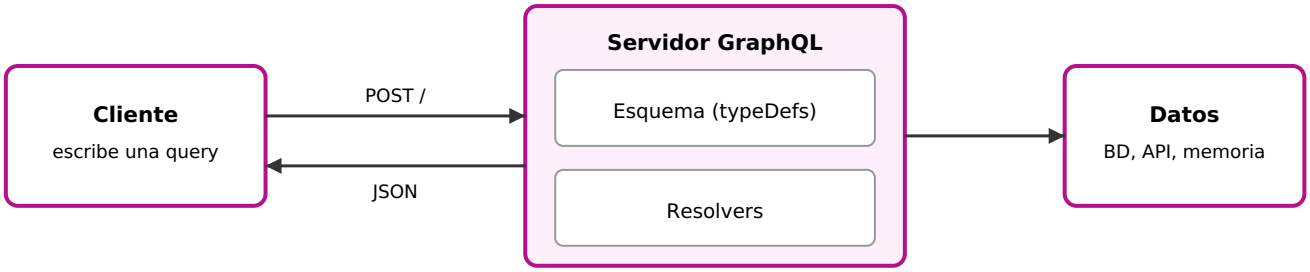
2.3 Quién lo mantiene y versión actual

- **GraphQL (la especificación):** la gobierna la **GraphQL Foundation**, alojada en la Linux Foundation. La edición vigente es la de **septiembre de 2025**, la primera desde la de octubre de 2021.
- **Apollo Server:** es la implementación de servidor que uso en este manual. La mantiene la empresa Apollo GraphQL. La versión actual es la **5** (la 4 dejó de recibir soporte el 26 de enero de 2026).

Hay que distinguir entre GraphQL, que es la especificación, y las implementaciones, como Apollo Server, que son las librerías que la cumplen. Existen implementaciones para casi todos los lenguajes.

3. Conceptos clave

Estas son las seis ideas que conviene tener claras antes de escribir código. Todas aparecen en el ejemplo del apartado 5.



Concepto	Qué es
1. Esquema (schema)	El contrato de la API: describe qué tipos de datos existen y qué operaciones se pueden hacer. Se escribe en el lenguaje SDL. Cliente y servidor se ponen de acuerdo a partir de él.
2. Tipos	Los bloques del esquema. Hay objetos (`Libro`), escalares (`String`, `Int`, `Boolean`, `ID`), listas (`[Libro]`) y valores obligatorios con `!` (`String!` significa "nunca nulo").
3. Query	Operación de **lectura**. El cliente indica los campos que quiere y recibe solo esos.
4. Mutation	Operación de **escritura** (crear, modificar, borrar). Tiene la misma sintaxis que una query, pero deja claro que cambia datos.
5. Resolver	La función que obtiene el valor de un campo. El esquema dice *qué* existe; el resolver dice *cómo* conseguirlo.
6. Introspección	Un servidor GraphQL puede describir su propio esquema. Gracias a eso existen herramientas como Apollo Sandbox, que ofrece autocompletado y documentación automática.

Ejemplo mínimo

Query (lo que pide el cliente)

```
{
  libro(id: "1") {
    titulo
    anio
  }
}
```

Respuesta (misma forma)

```
{
  "data": {
    "libro": {
      "titulo": "Cien años de soledad",
      "anio": 1967
    }
  }
}
```

Aunque `Libro` tiene más campos (`leido`, `autor`…), solo llegan los dos que se han pedido.

4. Requisitos e instalación

4.1 Requisitos previos

- **Node.js 20 o superior** (requisito de Apollo Server 5). Se comprueba con `node --version`.
- **npm**, que viene incluido con Node.js.
- **Editor de código**, por ejemplo Visual Studio Code.
- **Navegador** para abrir Apollo Sandbox. No hace falta ninguna cuenta ni servicio externo.

4.2 Instalación paso a paso

```
# 1. Crear la carpeta del proyecto y entrar en ella
mkdir ejemplo
cd ejemplo

# 2. Inicializar el proyecto y activar módulos ES (import/export)
npm init --yes
npm pkg set type="module"

# 3. Instalar Apollo Server y la librería base de GraphQL
npm install @apollo/server graphql

# 4. Crear los archivos del código (ver apartado 5) y arrancar
npm start
```

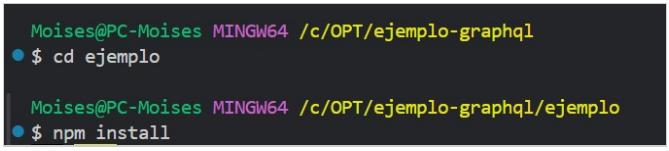


Figura 1. Instalación de las dependencias con «npm install» dentro de la carpeta del proyecto.

Antes de arrancar hay que añadir en el `package.json` el script `"start": "node src/index.js"`. Si el servidor arranca bien, muestra en la terminal: `Servidor GraphQL listo en http://localhost:4000/`.

4.3 Estructura de carpetas

```
ejemplo/
├── package.json        # dependencias y script "start"
├── package-lock.json   # versiones exactas instaladas (lo genera npm)
├── node_modules/       # librerías descargadas (no se sube a GitHub)
└── src/
    ├── index.js        # crea y arranca el servidor Apollo
    ├── schema.js       # esquema GraphQL (tipos, queries, mutations)
    ├── resolvers.js    # funciones que devuelven los datos
    └── data.js         # datos de ejemplo en memoria
```

4.4 Capturas de que funciona

Tras «npm start», al abrir `http://localhost:4000` aparece Apollo Sandbox, una herramienta del propio Apollo Server para escribir consultas y ver la documentación del esquema. En el panel izquierdo se ve que el esquema tiene un tipo `Query` y otro `Mutation`.



Figura 2. El servidor arranca con «npm start» y escucha en http://localhost:4000/.

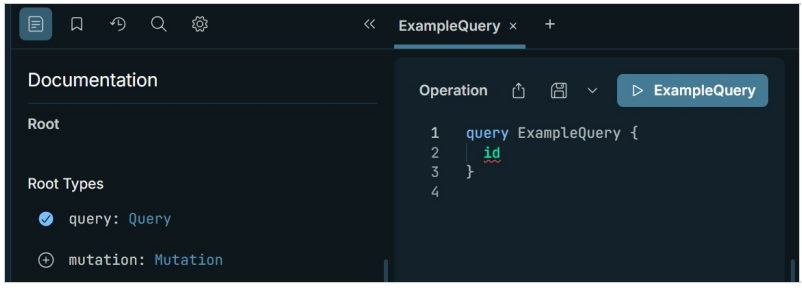


Figura 3. Apollo Sandbox en http://localhost:4000, con los tipos raíz Query y Mutation del esquema.

5. Ejemplo básico de uso: catálogo de libros

5.1 El miniproyecto

Es una API GraphQL para gestionar un catálogo de libros y autores. Permite consultar libros (con filtro de «no leídos»), consultar autores con sus libros, añadir un libro nuevo y marcar un libro como leído. Los datos viven en memoria, así que al reiniciar el servidor vuelven a su estado inicial. Así el ejemplo se centra en GraphQL y no en bases de datos.

5.2 Código explicado por partes

Parte 1: datos de ejemplo (`src/data.js`)

```
export const autores = [
  { id: "1", nombre: "Gabriel García Márquez" },
  { id: "2", nombre: "Isabel Allende" },
  { id: "3", nombre: "Julio Cortázar" },
];

export const libros = [
  { id: "1", titulo: "Cien años de soledad", anio: 1967, leido: true, autorId: "1" },
  { id: "2", titulo: "La casa de los espíritus", anio: 1982, leido: false, autorId: "2" },
  { id: "3", titulo: "Rayuela", anio: 1963, leido: false, autorId: "3" },
  { id: "4", titulo: "El amor en los tiempos del cólera", anio: 1985, leido: false, autorId: "1" },
];
```

Dos arrays de objetos. Cada libro guarda el `autorId` de su autor, como una clave foránea.

Parte 2: el esquema (`src/schema.js`)

```
export const typeDefs = `#graphql
  type Autor {
    id: ID!
    nombre: String!
    libros: [Libro!]!      # un autor tiene varios libros
  }

  type Libro {
    id: ID!
    titulo: String!
    anio: Int
    leido: Boolean!
    autor: Autor!          # un libro tiene un autor
  }

  type Query {             # operaciones de LECTURA
    libros(soloNoLeidos: Boolean): [Libro!]!
    libro(id: ID!): Libro
    autores: [Autor!]!
  }

  type Mutation {          # operaciones de ESCRITURA
    anadirLibro(titulo: String!, anio: Int, autorId: ID!): Libro!
    marcarLeido(id: ID!): Libro
  }
`;
```

Aquí se define el contrato. Fíjate en que `Libro` y `Autor` se referencian entre sí: eso permite al cliente navegar el «grafo» de datos, que da nombre a la tecnología. El `!` marca los campos que nunca son nulos; `anio` no lo lleva, así que es opcional.

Parte 3: los resolvers (`src/resolvers.js`)

```
import { GraphQLError } from "graphql";
import { autores, libros } from "./data.js";

export const resolvers = {
  Query: {
    // (padre, argumentos) => valor
    libros: (_padre, { soloNoLeidos }) =>
      soloNoLeidos ? libros.filter((l) => !l.leido) : libros,
    libro: (_padre, { id }) => libros.find((l) => l.id === id) ?? null,
    autores: () => autores,
  },

  Mutation: {
    anadirLibro: (_padre, { titulo, anio, autorId }) => {
      if (!autores.some((a) => a.id === autorId)) {
        throw new GraphQLError(`No existe ningún autor con id ${autorId}`, {
          extensions: { code: "BAD_USER_INPUT" },
        });
      }
      const nuevo = { id: String(libros.length + 1), titulo,
                      anio: anio ?? null, leido: false, autorId };
      libros.push(nuevo);
      return nuevo;
    },
    marcarLeido: (_padre, { id }) => {
      const libro = libros.find((l) => l.id === id);
      if (!libro) return null;
      libro.leido = true;
      return libro;
    },
  },

  // Resolvers de relaciones: solo se ejecutan si el cliente pide ese campo
  Libro: { autor: (libro) => autores.find((a) => a.id === libro.autorId) },
  Autor: { libros: (autor) => libros.filter((l) => l.autorId === autor.id) },
};
```

Cada campo del esquema tiene su función. Los resolvers de `Libro.autor` y `Autor.libros` son los que resuelven las relaciones, y GraphQL solo los llama cuando la query pide esos campos. Si `anadirLibro` recibe un autor que no existe, devuelve un error GraphQL en lugar de guardar datos incorrectos.

Parte 4: arrancar el servidor (`src/index.js`)

```
import { ApolloServer } from "@apollo/server";
import { startStandaloneServer } from "@apollo/server/standalone";
import { typeDefs } from "./schema.js";
import { resolvers } from "./resolvers.js";

const server = new ApolloServer({ typeDefs, resolvers });  // esquema + resolvers

const { url } = await startStandaloneServer(server, { listen: { port: 4000 } });
console.log(`Servidor GraphQL listo en ${url}`);
```

Un servidor Apollo no es más que el esquema unido a sus resolvers. `startStandaloneServer` levanta un servidor HTTP con la configuración mínima.

5.3 Probar el servidor: queries y mutations

Estas operaciones se pegan en el panel *Operation* de Apollo Sandbox y se ejecutan con el botón azul.

a) Query: libros con su autor

```
query {
  libros {
    titulo
    anio
    autor { nombre }
  }
}
```

Devuelve los libros del catálogo, cada uno con el nombre de su autor, en una sola petición. Con REST habrían hecho falta varias.



Figura 4. Respuesta de la query (primeros libros de la lista).

b) Query con argumento: solo los no leídos

```
query {
  libros(soloNoLeidos: true) { id titulo }
}
```

Debe devolver tres libros, porque «Cien años de soledad» está marcado como leído.

c) Mutation: añadir un libro

```
mutation {
  anadirLibro(titulo: "Ficciones",
              anio: 1944, autorId: "3") {
    id
    titulo
    autor { nombre }
  }
}
```

Crea el libro y devuelve el resultado con los campos pedidos: recibe el `id` 5 y queda asociado a Julio Cortázar. Si se repite la query del apartado a), el libro nuevo aparece en la lista.



Figura 5. Respuesta de la mutation `anadirLibro`.

d) Mutation con error controlado

```
mutation {
  anadirLibro(titulo: "Prueba",
              autorId: "99") { id }
}
```

Como el autor 99 no existe, el resolver lanza un `GraphQLError` y el servidor responde con el mensaje «No existe ningún autor con id 99» y el código `BAD_USER_INPUT`, sin guardar nada.



Figura 6. Respuesta con el error controlado.

6. Ventajas, inconvenientes y conclusión

Ventajas

- El cliente pide exactamente lo que necesita: menos datos y menos peticiones.
- Un único endpoint para toda la API.
- El esquema tipado hace de documentación y de contrato; las herramientas dan autocompletado y detectan errores.
- Permite evolucionar la API añadiendo campos sin romper a los clientes existentes.
- Ecosistema grande y mantenido por una fundación neutral.

Inconvenientes

- Curva de aprendizaje: hay que entender esquema, resolvers y un lenguaje de consulta nuevo.
- El cacheo HTTP es más difícil que en REST, porque casi todo viaja por un único `POST`.
- Una query muy anidada puede ser costosa; en producción hay que limitar la profundidad y vigilar el problema de las «N+1 consultas».
- Para una API pequeña y simple puede ser más complejo de lo necesario.

Conclusión y valoración personal

GraphQL me ha parecido más fácil de empezar de lo que esperaba: con un esquema, cuatro resolvers y menos de veinte líneas para arrancar el servidor, ya tenía una API que se puede explorar desde el navegador. Lo que más me ha gustado es que el esquema funciona como documentación viva, y que una sola consulta trae datos relacionados sin encadenar peticiones.

Lo usaría en un proyecto real cuando haya **varios clientes distintos** (web y móvil, por ejemplo) con necesidades de datos diferentes, o cuando los datos estén muy relacionados entre sí, como una red social, una tienda con productos, categorías y reseñas, o un panel con muchas pantallas. Para una API pequeña con pocas rutas, REST seguiría siendo la opción más sencilla.

7. Recursos y bibliografía

- GraphQL Foundation. *Learn GraphQL* (documentación oficial): `https://graphql.org/learn/`
- GraphQL Foundation. *Especificación de GraphQL, edición de septiembre de 2025*: `https://spec.graphql.org/September2025/`
- GraphQL Foundation. *Preguntas frecuentes generales*: `https://graphql.org/faq/general/`
- Apollo GraphQL. *Apollo Server: Get started*: `https://www.apollographql.com/docs/apollo-server/getting-started`
- Apollo GraphQL. *Previous versions of Apollo Server* (versiones y fin de soporte): `https://www.apollographql.com/docs/apollo-server/previous-versions`
- Node.js. *Descarga e instalación*: `https://nodejs.org`
- Repositorio de este manual y del ejemplo: https://github.com/MoisesFlorezDAW/manual-graphql-florez